

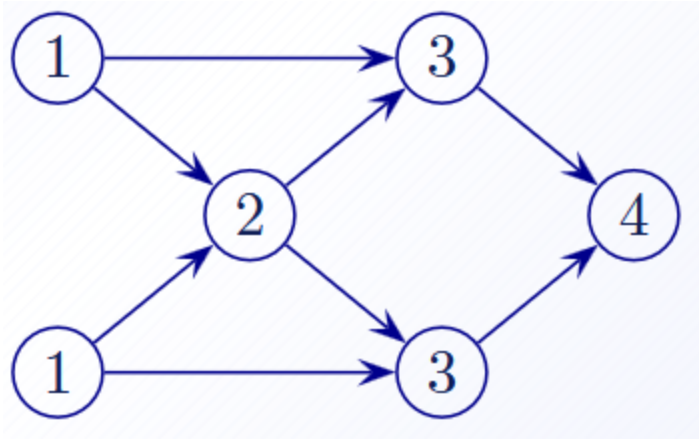
TE 2

Chap 6 (Graph sans circuits)

[x6.1, x6.2, x6.4, x6.7, x6.10, 6.11, x6.13, 6.9, 6.14, 6.15, 6.12]

Definition

Un graph sans circuit est défini si par posséder au moins un sommet sans prédécesseur et un sommet sans successeur. Tous les sous-graphes partiels doivent être aussi sans circuit.



Fonction de rang

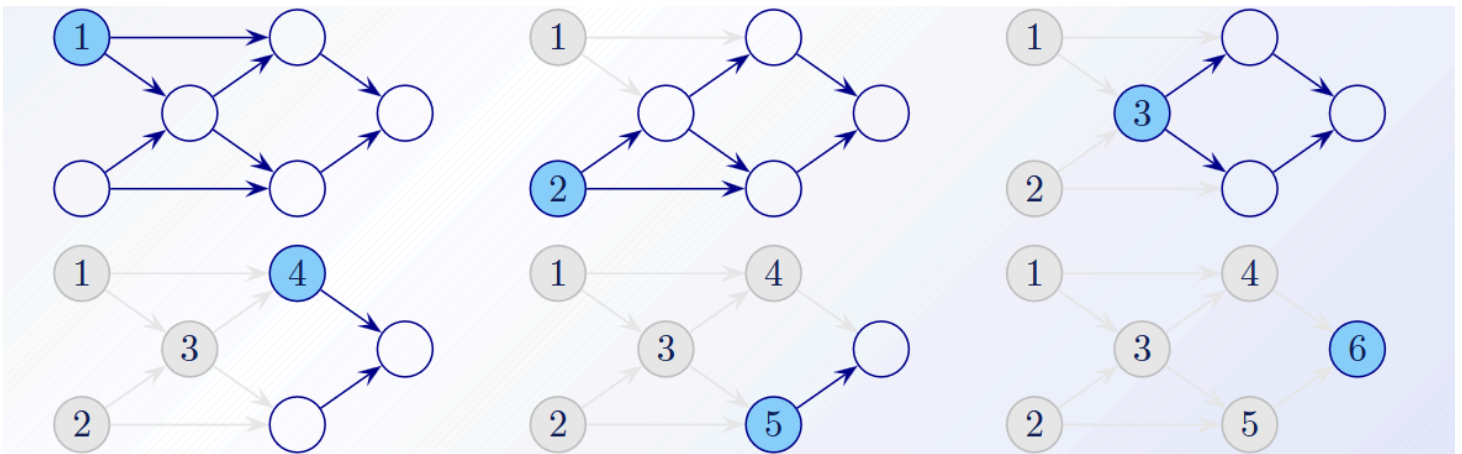
C'est une fonction qui permet de numérotée les sommets du graphe (tri topologique)

Un graph sans circuit est un graph qui admet une fonction de rang

Tri topologique

Tant qu'il reste des sommets non numérotés faire

- Identifier un sommet sans prédécesseurs
- Le numéroté (à la suite de celui de l'itération précédente)
- Le supprimer du graphe

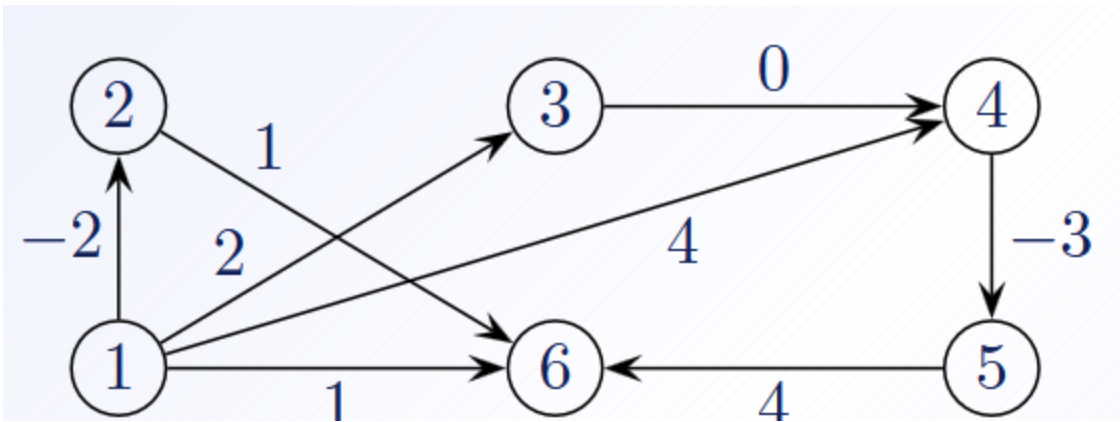


Complexité : $O(N+M)$

Théorème. Les dates de fin de traitement obtenues lors de l'exploration en profondeur d'un graphe sans circuits G définissent une fonction de rang pour le graphe transposé G^T .

Chemin le plus court

Vue qu'on n'a pas de circuit, on peut just appliquer Bellman, faut juste traiter les sommet par **numérotation topologic**



Somet traiter	1	2	3	4	5	6
1	-(0)	----	----	----	----	----
2	-(0)	1(-2)	----	----	----	----
3	-(0)	1(-2)	1(2)	----	----	----
4	-(0)	1(-2)	1(2)	3(2)	----	----
5	-(0)	1(-2)	1(2)	3(2)	4(-1)	----
6	-(0)	1(-2)	1(2)	3(2)	4(-1)	2(-1)

pred (distance)

Algorithme PCC_SANSCIRCUITS

- Tri topologic de $G \rightarrow G'$
- BellMan sur G' (source sommet #1)

Complexité : $O(N+M)$

Problèmes de plus longs chemins

Vue qu'on a un graph sans cercuit, on s'en fous des cercuit à coût negatif ou à coût positif

Donc on peut just fair Bellman mais avec un *max* à la place du *min*

Application

Problème central de l'ordonnancement

- Un ensemble de n tâches doivent être réalisées.
- Pour chaque tâche i on connaît sa durée di d'exécution.
- La planification doit respecter des contraintes d'antériorité du type :
 - La tâche j ne peut commencer qu'une fois l'exécution de la tâche i terminée.
- L'objectif est de déterminer une planification des tâches respectant toutes les contraintes et minimisant la durée totale de réalisation

Les graphes potentiels-ÉTAPES

les sommets du graphe représentent différentes étapes de réalisation du projet global et les arcs correspondent aux tâches

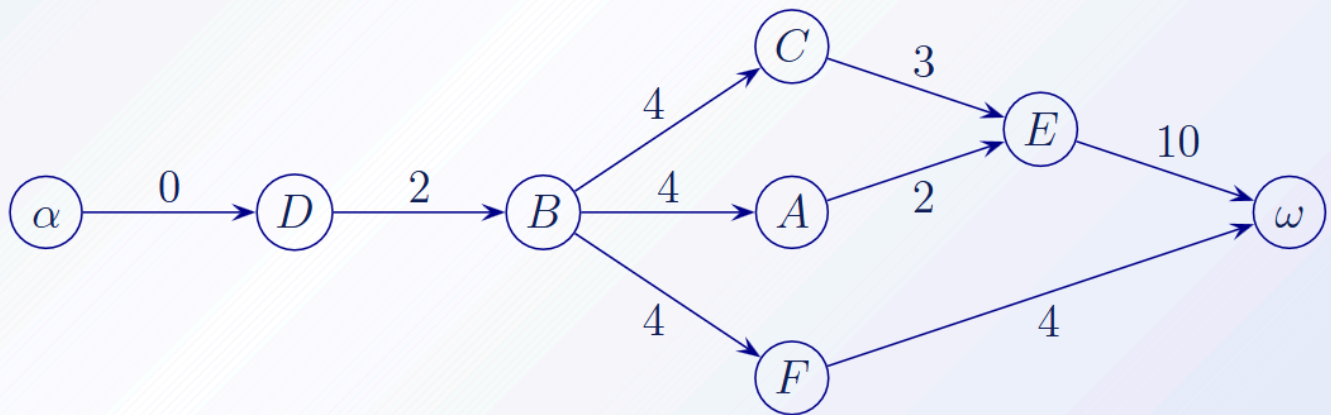
Les graphes potentiels-TÂCHES

sommet du graphe correspond à une tâche et les arcs modélisent les contraintes d'antériorité

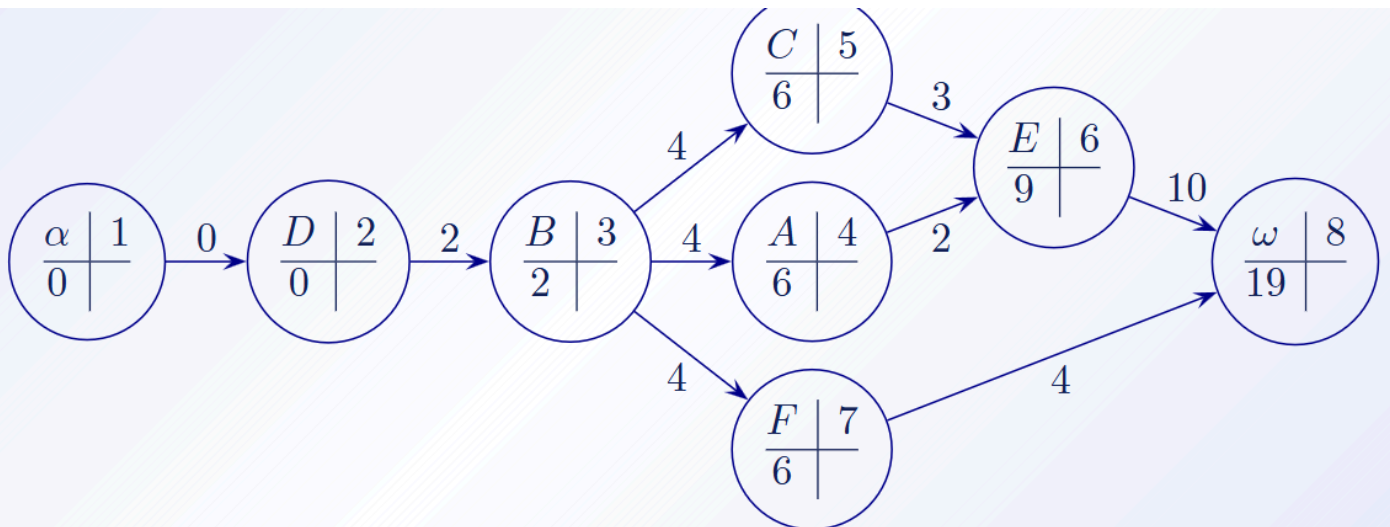
Pour le modélisé, c'est trivial

Faut just pas oublier de rajouter un sommet *début* d'où toutes les taches parte et un sommet *fin* qui représent la fin du projet et qui n'a donc pas de successeur

Tâche	Description	Durée (sem.)	Antériorités
D	Choix des stations	2	—
B	Accord administratif	4	D
C	Commande des décodeurs	3	B
A	Installation des antennes	2	B
E	Installation des décodeurs	10	C,A
F	Modification de la facturation	4	B

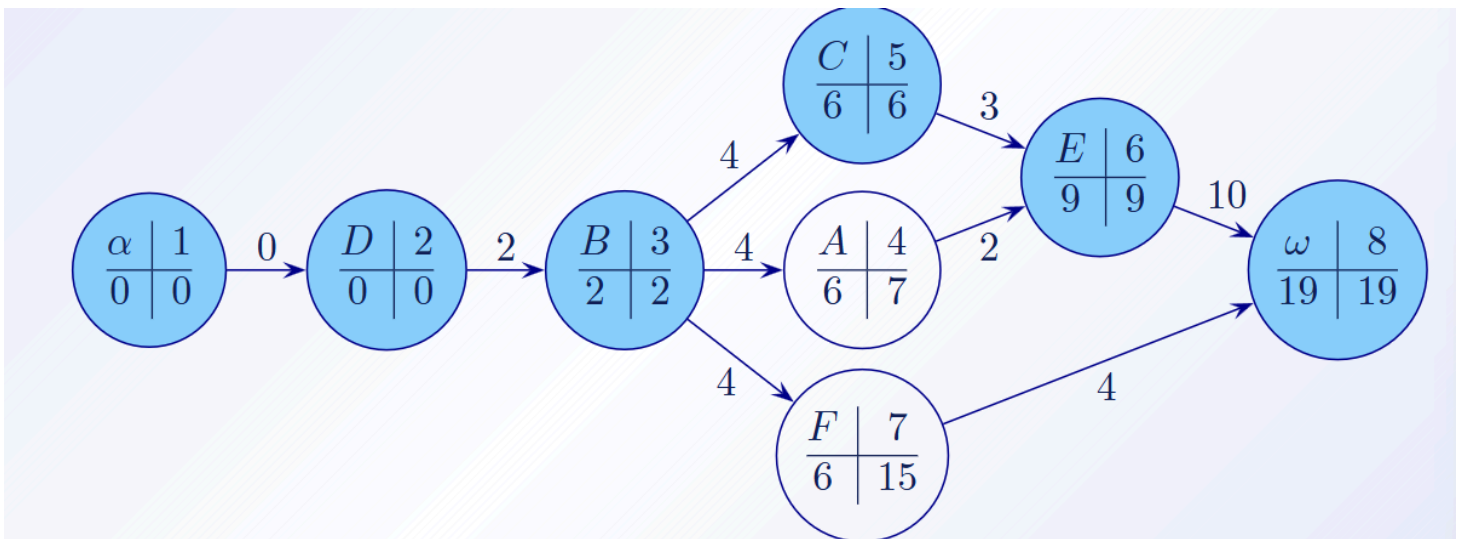


La date de **debut de réalisation** d'une tâche est donc donner par son chemin le **plus long**



Chemin critique

Un chemin critique est un plus long chemin du sommet α (début des travaux) au sommet ω (fin des travaux) (il est donc composé uniquement de tâches critiques). La longueur D d'un chemin critique correspond à la durée minimale nécessaire à la réalisation du projet.



(le petit nombre a droite est égale la date de debut de relisation du la tache précédent - duré la tache, sois la date maximal pour commencer la tache)

Chap 7 (flots)

[x7.1, 7.2, 7.4, 7.5, 7.8, 7.9, 7.10, 7.12, 7.13, 7.14, 7.15, 7.16, 7.17, 7.18, 7.19]

Important

dans un flot, y a toujours :

- une source S (sommet avec 0 pred)
- un puits T (sommet sans succ)
- une fonction U qui associe chaque arc du graph a une capacité maximal nomée x_{ji} ou j et i sont les id des sommet de l'arc

Le but est de transmettre un quantité maximal de "truc" sans perte depuis la source jusqu'au puit

Équation de conservation du flot

- flux entrant = $\sum$ des x_{ij} des *pred* du sommet
- flux sortant = $\sum$ des x_{ij} des *succ* du sommet
- conservation du flot = $\sum pred - \sum succ$
le but est d'avoir un equoation du flot = 0

Flot Compatible

Un flot compatible c'est un flot qui respect l'équation de conservation

Algorithme de Ford et Fulkerson

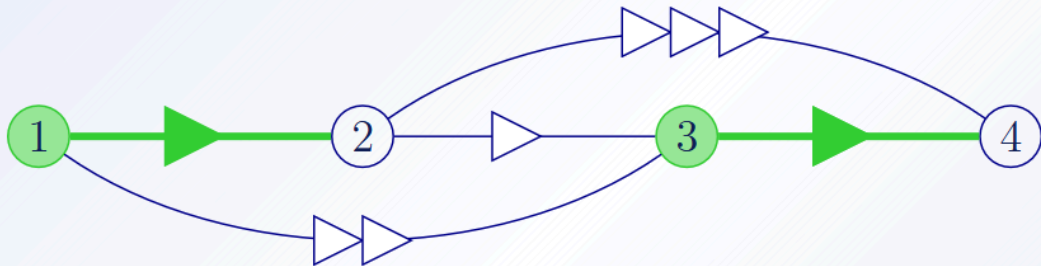
Pas polinomial, du au fait que suivant les choix de recherche, on peut avoir + ou - d'iteration

1. prendre un flot initial possible
2. Crée R^* (réseau d'augmentation)
3. chercher le chemin de s à t dans R^*
4. si trouver, augmenter au maximum le flot le long de ce chemin (en diminuant les flot inverse)
5. répéter jusqu'à qu'on ai plus de chemin dans R^*

Coupe

En gros, l'idée c'est de séparé les sommet en 1 group A, et de faire en sort que chaque arc du group A vas en dehors de celui ci

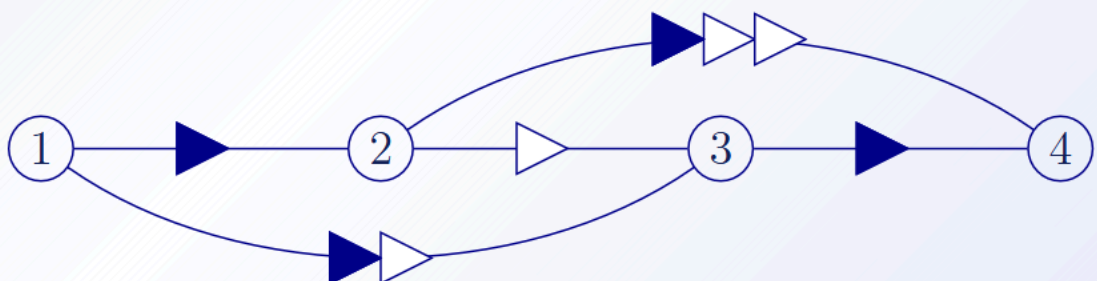
La coupe définie par l'ensemble $A = \{1, 3\}$ est formée des arcs $(1, 2)$ et $(3, 4)$:



La capacité de cette coupe est defini par la somme des capacité des arc formant la coupe

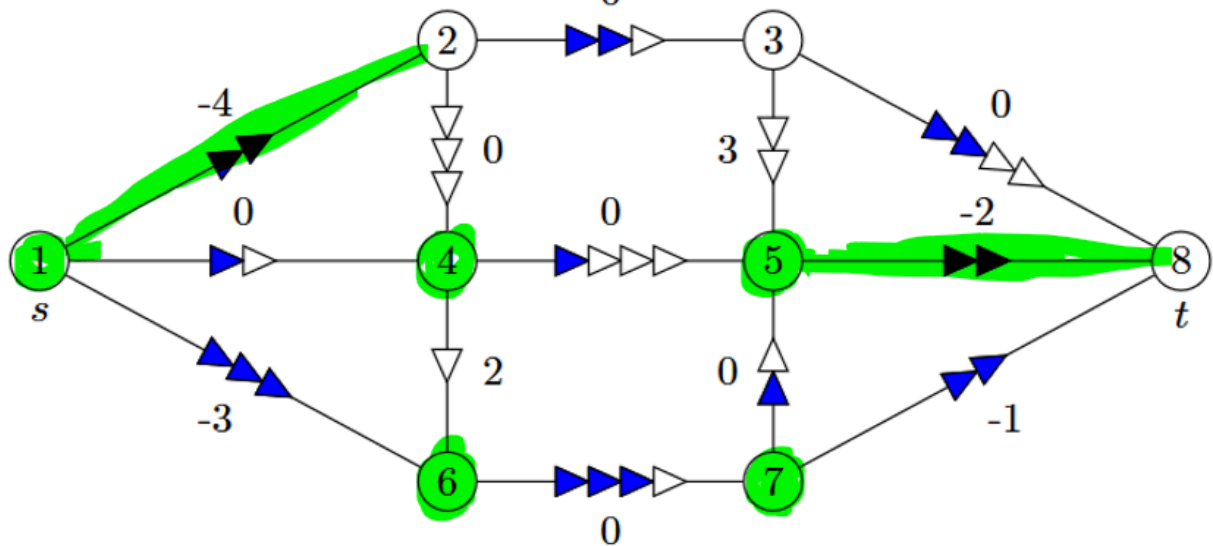
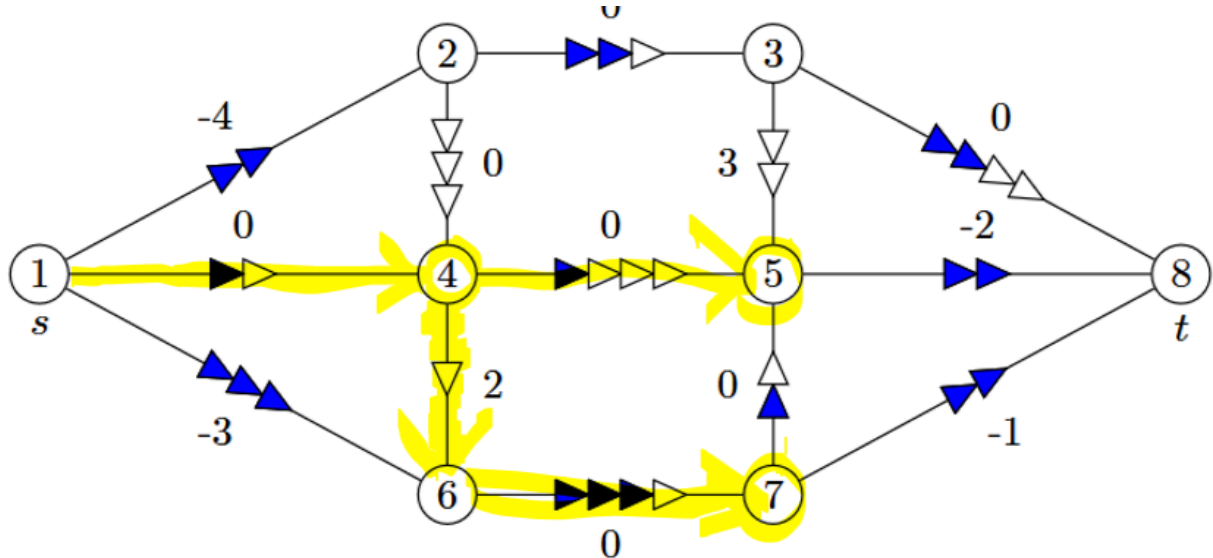
dans cette exemple, on a $A = \{1, 3\}$ définie par les arc $(1, 2)$ et $(3, 4)$ => donc une capacité de 2

si notre flow est égale a la capacité de la coupe, alors la coupe est minimal et le flot est maximal



pour le trouver,

- tu prend la dernier explo
- tu check tous les sommet que tu atteindre depuis la source
- tu prend chacun de ses sommet et fait ta coupe avec les flot plein



Théorème du flot max et de la coupe min

Dans un réseau, la valeur du flot max de S à T est toujours égale à la capacité minimal d'une coupe séparant s de t

Couplage maximum dans un graphe biparti

Dans un cas où le graph est bipartie, on peut voir le problème comme une résolution de couplage max dans un graphe bipartie.

en gros, on prend notre graph :

- on oriente tous les arc d'un groupe vers l'autre
- on met la capacité de chaque arc à 1
- on ajoute une source "s" et un puits "t"
- on cherche le flot max de s à t

Problème a flot max à coût (\$) min

en gros, on met un coût de n par unité sur les arcs, dans ce réseau on cherche le flot max mais à coût min

Algorithme de Busacker et Gowenver coût

1. on commence avec un flot initial null
2. Construire un réseau d'augmentation R^*
3. Cherche un plus court chemin de s à t dans R^*
 - si trouver, répéter depuis (2)
 - (si coût) mettre à jour les coûts en faisant $c_{ij} = c_{ij} + (\lambda_i - \lambda_j)$ (pour éviter les coûts négatifs)
4. end

Problème d'affectation linéaire et transbordement

on a N personnes avec N tâches et on cherche comment les répartir de manière qu'on ait une tâche par personne.

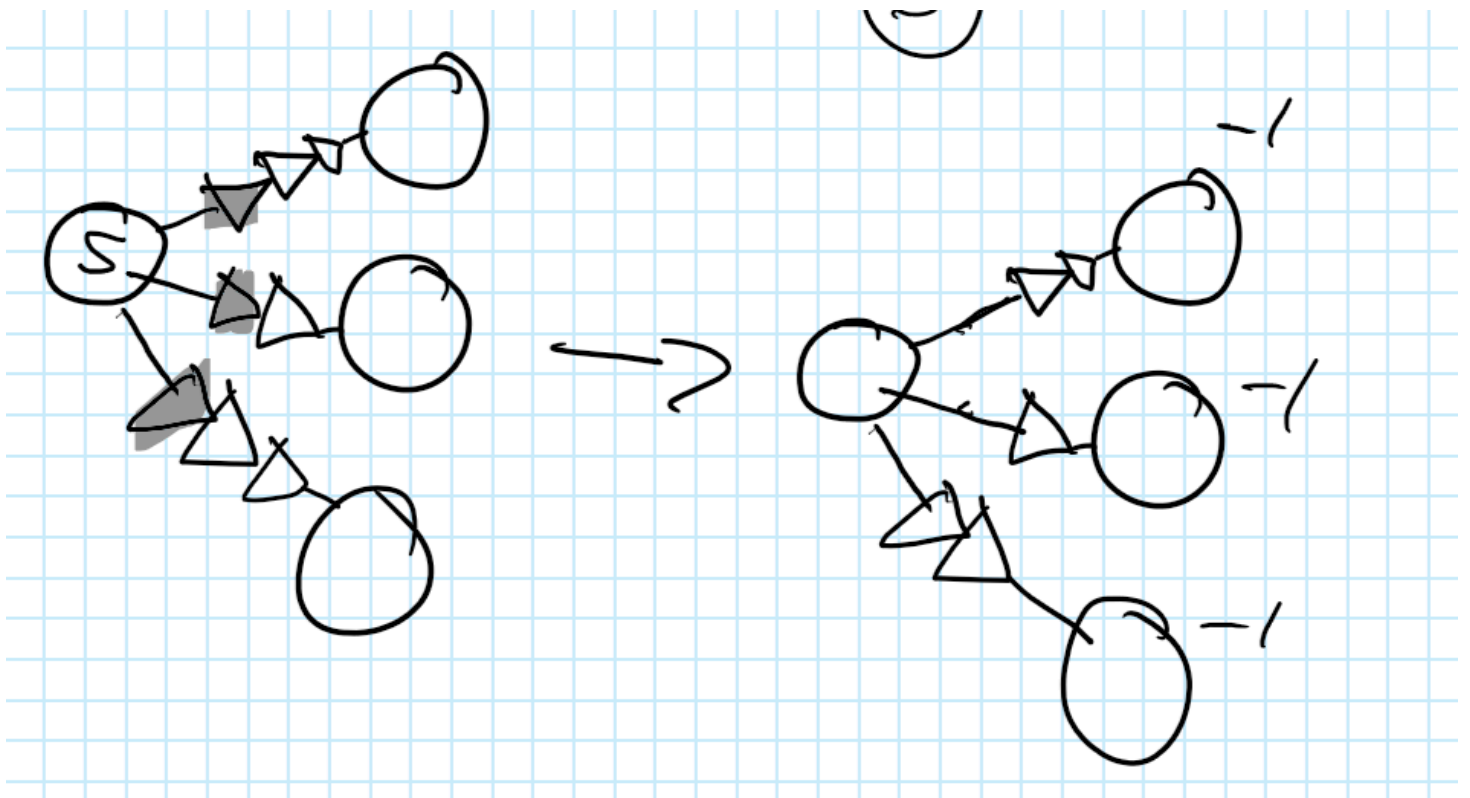
En gros, c'est problème de couplage parfait de coût minimum dans un graphe biparti.

donc on a :

- des sources avec N unités
- des puits avec qui doivent demander le même nombre d'unités
- et des sommets de transit avec ni offre ni demande

BTW, un puits c'est pas seulement un sommet sans successeur, mais bien à partir du moment où un sommet consomme quelque chose

Éliminé un flow a capacité min



Definition d'un problème

- donner ce que l'on cherche
- définir les différentes parties de notre modélisation
- définir comment savoir si on a résolu le problème

Other shit

- on a un réseau R avec une *fnc* associant chaque sommet une capacité
 - splitter chaque sommet en 2 (sortant et entrant), mettre l'arc entre entrant et sortant à la capacité *fnc*, résoudre le problème de flow
- Graph G , trouve le nombre max de chemins de s à t disjoints par
 - arc, on met une capacité de 1 par arc into flow max
 - sommet, on met une capacité de 1 par sommet into flow max en utilisant la technique ci-dessus
- trouver le plus court chemin dans un graph à pondération
 - arc de s et de t on met une capacité de 1, les autres arcs auront aussi une capacité de 1 et aussi le même coût unitaire que la pondération, into flowmax
- trouver arbre min
 - Source s avec une capacité de $|V|-1$ et tous les autres sommets ont une demande 1 puis même logique

Chap 8 (Random problème)

[8.1, 8.2, 8.3, 8.4, 8.5, 8.7, 8.8, -8.9, -8.10, -8.11, -8.12]

Graph complet

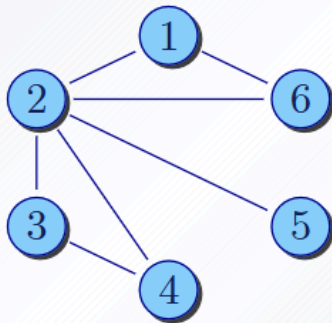
Graph K_n compte $\frac{n(n-1)}{2}$ arêtes

Graph complémentaire

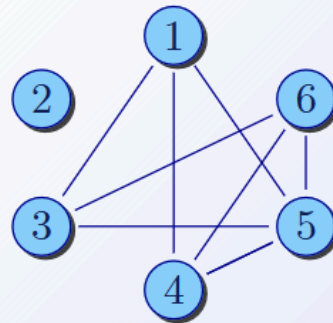
c'est le graph qui complet (d'un point de vue arête) le graph G

ex :

EXEMPLE.



Graphe G

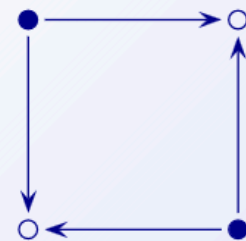
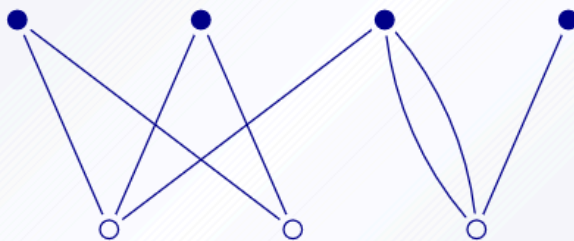


Graphe complémentaire $\overline{G}$

Graph bipartie (bi-partie)

En gros, c'est un graph où l'on catégorise l'ensemble des Edge en 2 groups et dans le quel chaque Edge est relia par une Vertex a un Edge de l'autre groupe

EXEMPLES.



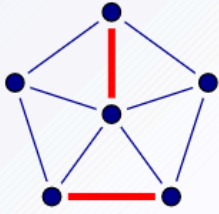
Graph bipartie complet

c'est un graph bipartie dans le quel chaque Edge d'un group est relia a tous les autre Edge de l'autre group

- Recouvrement = sous-ensemble d'arc tel que chaque sommet de G soit au moins une extrémité des arcs
- transversal = sous-ensemble d'arc

Couplage

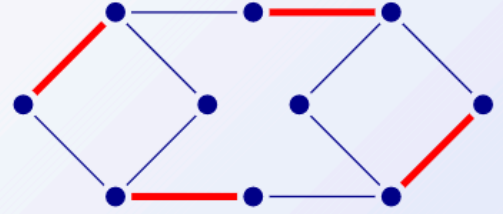
EXEMPLES.



Couplage non maximum
(mais maximal / inclusion)



Couplage parfait
et donc maximum



Couplage maximum
mais pas parfait

- Parfait = chaque Edge est coupler avec un autre
- Maximum = en gros, c'est quand on utilise la maximum d'arc et que l'on ne peut plus en rajouter

Chain

- Chaîne alternée relative à M : chaîne dans l'aténér classic



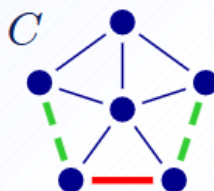
- Chaîne alternée augmentante à M : si y a les deux extrémités de la chaîne qui ne sont pas dans M



Théorème de Berge

en gros, il dit que si on a une chaîne donnée C, on peut lui ajouter un membre en faisant :

- crée un couplage d'arc non compris dans C
- faire $C \Delta M$



$$M' = M \Delta C$$

Recherche de couplage maximum

1. crée un couplage, peut importe le quel
2. Oriente les arc :
droite $\rightarrow$ gauche

et gauche $\rightarrow$ droite

3. en partant du sommet "source" et allant vers un "pu" on obtient une chaîne alternée C augmentante sur M

4. on fait $C \Delta M$

Other shit

- graph bipartite $K_{r,s}$ ($r + s$ sommet)
 - $n_{\text{Arc}} = r * s$ / couplage max cardinalité = $\min(r, s)$
- Si on a le couplage max d'un graph G , trouver le recouvrement $\rightarrow$ ajoute les sommets manquants pour former l'arbre
- Si on a un arbre recouvrant min, trouver le couplage max = garder un arc pour tous les sommets à plus de 1 connexion